

Design Patterns — Prototype · Iterator · Singleton

UML class diagrams of the three design patterns actually implemented in the backend, drawn from the source in [Patterns/Prototype/](#), [Patterns/Iterator/](#) and [Patterns/Singleton/](#) — the same diagram style as the reference Interpreter engine.

LEGEND

Realization / implements



Dependency (uses)



Aggregation (has-a)

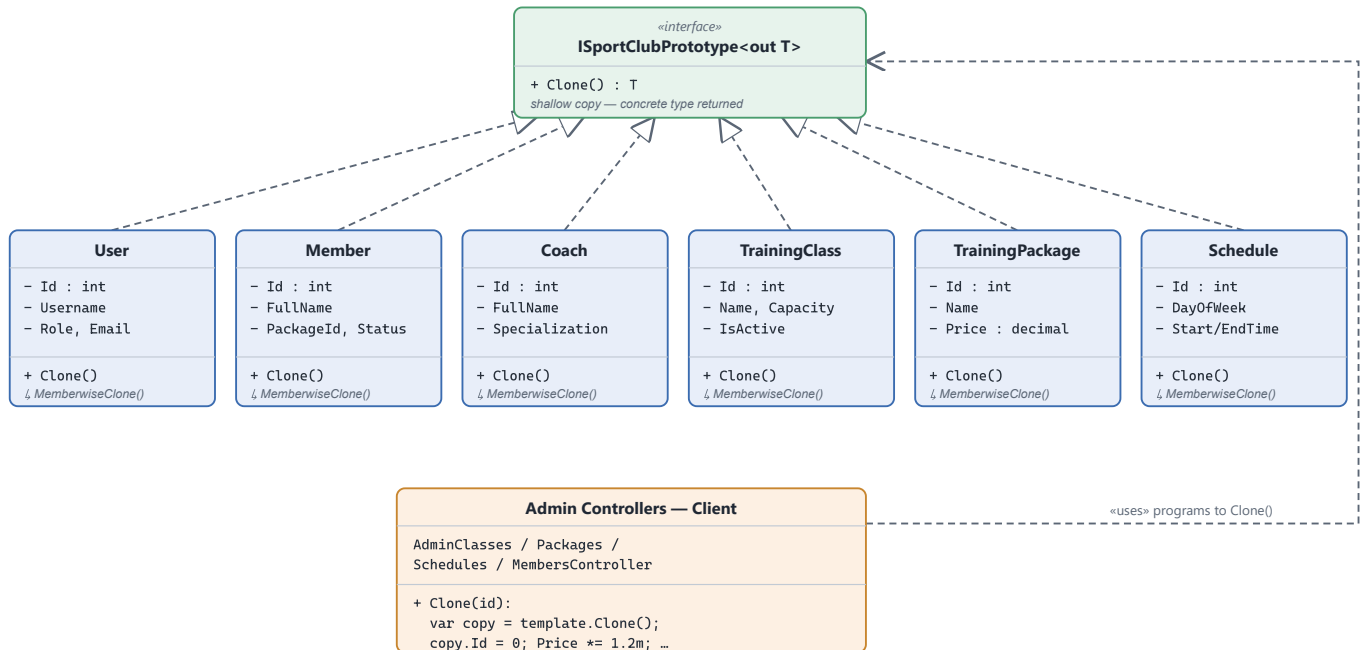


Composition (owns)

1 • Prototype Pattern

CREATIONAL — CLONE, DON'T RE-BUILD

Every cloneable domain object implements `ISportClubPrototype<T>.Clone()` with a shallow copy via `MemberwiseClone()`. The admin /clone endpoints duplicate a template entity, then tweak the copy (reset Id, rename, +20% price, etc.) instead of constructing it field-by-field — mirroring the original Java Cloneable root.



HOW IT WORKS

1. An admin requests `POST /.../{id}/clone`.
2. The controller loads the *template* entity from its repository.
3. `template.Clone()` returns a shallow copy (`MemberwiseClone()`).
4. The copy is tweaked: `Id = 0`, renamed, navigation refs cleared.
5. The repository inserts it as a brand-new row.

EXAMPLE — PACKAGE CLONE

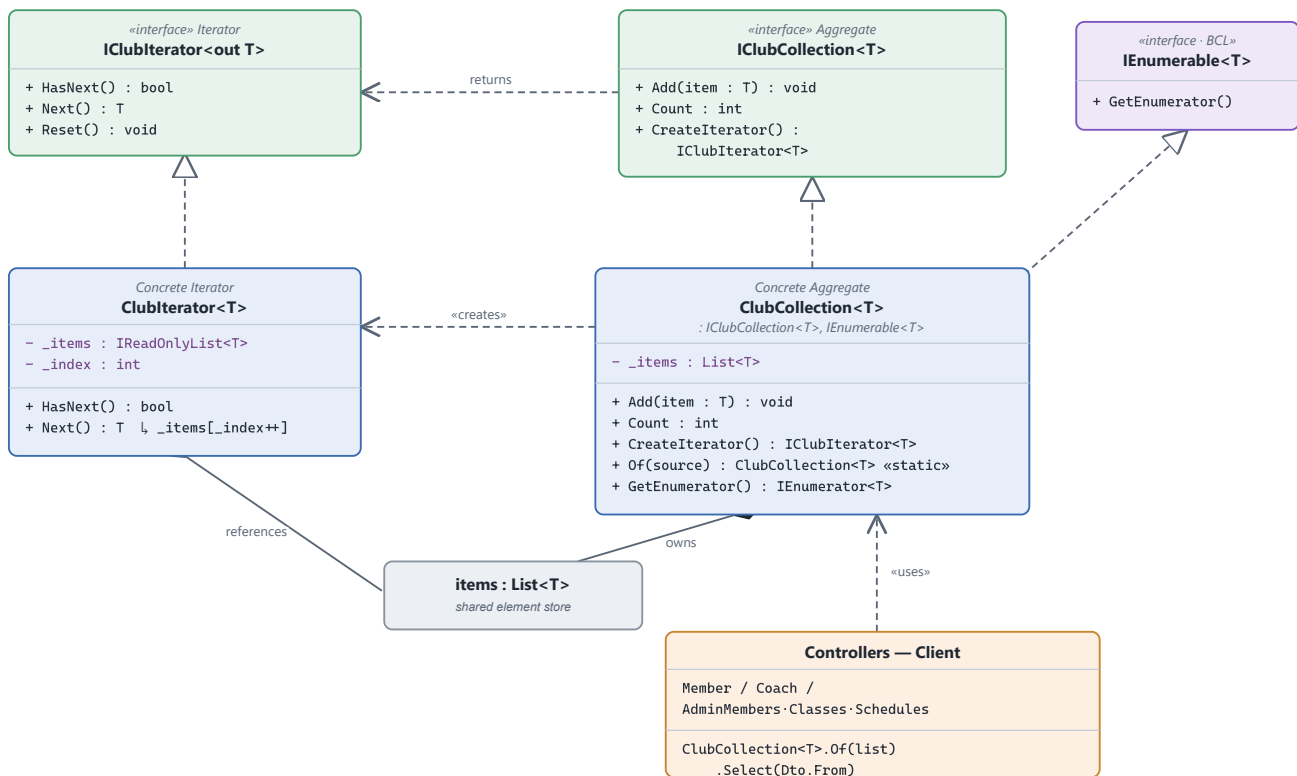
```
// AdminPackagesController.Clone
var copy = template.Clone();
copy.Id = 0;
copy.Name = "Copy of " + template.Name;
copy.Price = template.Price * 1.2m;
// 20% premium variant
await _packages.SaveChangesAsync(copy);
```

BENEFITS

- Duplicate complex entities without re-typing every field.
- One generic contract — add a prototype by implementing `Clone()`.
- Client never depends on a concrete constructor.
- Faithful to the original Java `Cloneable` design.

2 · Iterator Pattern BEHAVIORAL — TRAVERSE WITHOUT EXPOSING STORAGE

A single generic `ClubCollection<T>` wraps a `List<T>` and hands out a `ClubIterator<T>` (`HasNext` / `Next` / `Reset`). Because the collection also implements `IEnumerable<T>` — enumerating *through its own iterator* — list endpoints can call `ClubCollection.Of(src).Select(Dto.From)` and the projection still flows through the Iterator pattern.



HOW IT WORKS

1. A repository returns a raw `List<T>` of rows.
2. `ClubCollection<T>.Of(list)` wraps it as an aggregate.
3. `CreateIterator()` builds a `ClubIterator<T>` over the items.
4. `GetEnumerator()` walks it with `while (HasNext) Next()`.
5. LINQ `.Select(Dto.From)` consumes that enumerator — traversal stays inside the pattern.

EXAMPLE — LIST CLASSES

```
// AdminClassesController.List
var classes = ClubCollection<TrainingClass>
    .Of(await _classes.FindAllAsync());
```

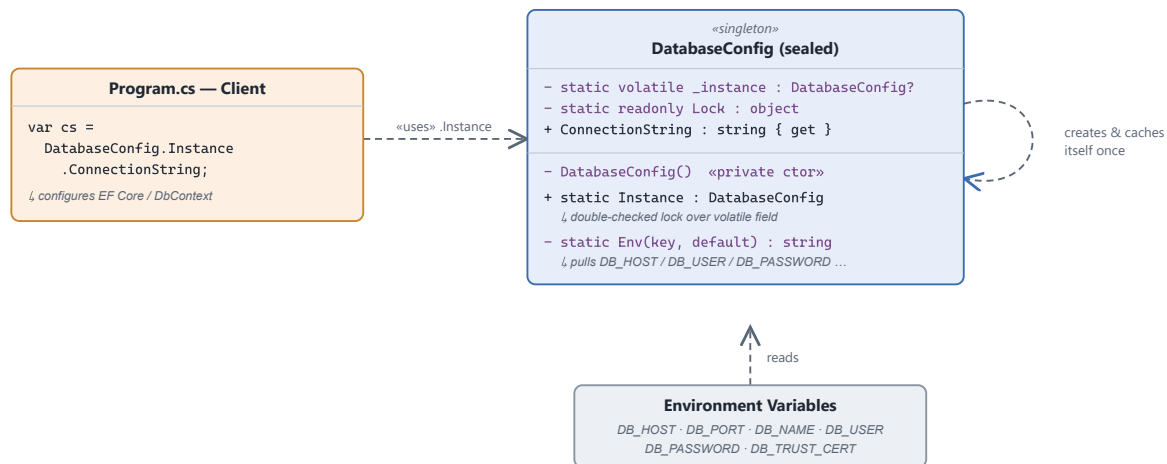
```
return Ok(classes.Select(ClassDto.From));  
// Select() pulls each item via  
// ClubIterator.HasNext()/Next()
```

BENEFITS

- Uniform traversal over any entity type — one generic collection.
- Backing storage (`List<T>`) stays hidden from callers.
- `IEnumerable<T>` bridges the pattern to standard LINQ.
- Replaces the per-entity Java collections with one type.

3 · Singleton Pattern CREATIONAL — EXACTLY ONE INSTANCE

DatabaseConfig assembles the SQL Server connection string *once* and shares it across the whole app. A private constructor plus **double-checked locking** over a volatile field guarantees a single, thread-safe instance. Credentials are read from environment variables (never hardcoded) — the security rationale for centralising them here. Program.cs pulls DatabaseConfig.Instance.ConnectionString to configure EF Core.



HOW IT WORKS

1. The constructor is `private` — no one can `new` it.
2. First `Instance` call enters the `lock` and creates the object.
3. `_instance` is `volatile` so all threads see it immediately.
4. Later calls skip the lock and return the cached instance.
5. The connection string is built once from env vars.

EXAMPLE — DOUBLE-CHECKED LOCK

```
public static DatabaseConfig Instance
{
    get {
        if (_instance is null)
            lock (Lock)
                _instance ??= new DatabaseConfig();
        return _instance;
    }
}
```

BENEFITS

- One authoritative connection string for the app.
- Thread-safe lazy init — built only when first needed.
- Credentials centralised & sourced from env vars, not code.
- Faithful to the Java `DatabaseConnection` singleton.